

MINI ASSESSMENT

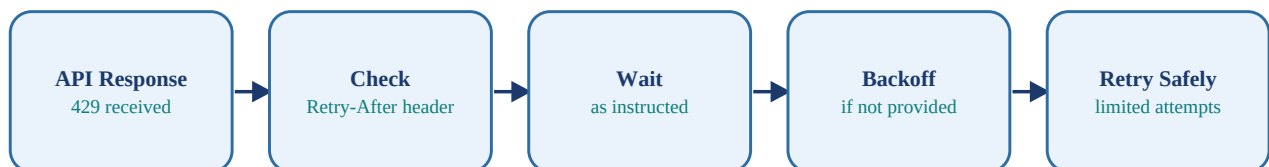
API Error Handling and Safe JSON Access

Name	KODIVALASA ASHIRASHREE
Reg. No.	99230040339
Department	CSE - DS

Question 1

If an API returns 429, what should your program do?

FLOW OF HANDLING A 429 RESPONSE



Answer

A **429 (Too Many Requests)** response means the client has exceeded the API's rate limit. The program should not keep sending requests immediately; instead it should pause and retry after a delay. If the server includes a **Retry-After** header, that exact wait time should be respected. If no such header is present, the program should apply an **exponential backoff** strategy (e.g., wait 1s, then 2s, then 4s, and so on) before retrying. To avoid an infinite loop, retries should be capped at a reasonable maximum, after which the failure should be logged and handled gracefully instead of crashing the program.

Question 2

If `response.json()` fails, what should your program do?

FLOW OF HANDLING A JSON PARSE FAILURE



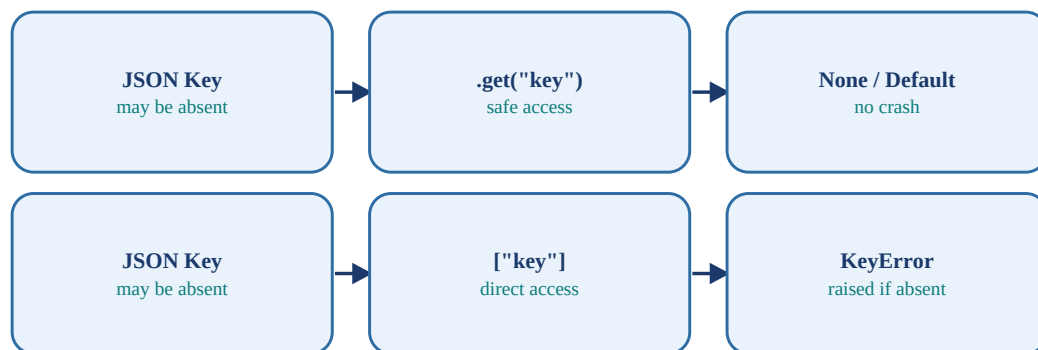
Answer

The program should **catch the JSON-decoding exception** (such as `json.JSONDecodeError`) in a try/except block so that a malformed or empty response does not crash the application. It is good practice to first check the HTTP status code, since a non-200 response often explains why the body is not valid JSON. Inspecting **response.text** can help with logging and debugging when parsing fails. Finally, the error should be handled with a clear, descriptive message, and the program should either continue safely with a fallback value or terminate that specific operation in a controlled way, rather than letting the whole application fail unexpectedly.

Question 3

If a JSON key is missing, why might `.get()` be safer than `[ ]`?

SAFE VS. UNSAFE KEY ACCESS



Answer

`data.get("key")` is safer than `data["key"]` when a key might be missing because `.get()` returns `None`, or a specified default value, instead of raising an exception. This allows the program to continue running even when the field is not present. In contrast, `data["key"]` assumes the key definitely exists and raises a `KeyError` if it is absent, which can crash the program if not explicitly handled. Therefore, `.get()` is the preferred choice for optional or uncertain fields in API responses.

QUICK COMPARISON

Method	Key Exists	Key Missing
<code>data.get("key")</code>	Returns value	Returns <code>None</code> / default
<code>data["key"]</code>	Returns value	Raises <code>KeyError</code>

KEY TAKEAWAY: Reliable API programs should handle rate limits, invalid responses, and missing JSON fields explicitly — this prevents unexpected crashes and makes the application resilient to real-world API behaviour.